Computer Graphics

CSCI 490, CSCI 631

Project 3

The Graphics Pipeline - Clipping and Depth Buffering

Summary

The purpose of this assignment is to introduce line clipping in x, y, and z into the graphics pipeline along with depth buffering of the final output. Some changes in the rendering of primitive objects will also be required.

Program Requirements

Your program should be able to

- clip points and lines in x, y and z with arbitrary near and far clipping planes.
- properly display depth ordering relationships between objects.
- more properly handle rendering of sphere objects.

Additional scene files are available for this project.

Things That Need to be Implemented and Modified

This is a list of data structures and routines that need to be implemented or modified.

Line Pipeline

- The line drawing pipeline should be modified to do line clipping. Clipping should take place in x, y and z. You may use any line clipping algorithm you wish, but the one given in class is recommended. A suggested implementation would be to put the clipping in a separate function. Then move everything in the line pipeline, from the point after the conversion to clip coordinates, into the line clipping routine. This means moving the static variables for the old vertex, the decisions about moving or drawing, the conversion to device coordinates and the homogeneous division are all moved to the clipping routine.

- Add a depth buffer to your renderer. The depth buffer should be allocated and initialized at the same time the image buffers are allocated and initialized. This should be done in in `rd_world_begin`.

  However, you don't want to dynamically allocate the depth buffer array each time you enter a world block. Make sure you have a test of some kind to see if the array has already been allocated. Allocate it only if needed.

  The buffer memory should be freed only at the end of the rendering process. It's inefficient to release it at the end of a world block only to allocate it again at the start of the next frame. The routine `rd_render_cleanup()` was designed for this purpose. It is called once by the renderer before exiting the program. Free up the depth buffer memory here.

- Because of the depth buffer, the line drawing routine must be modified. If your line drawing routine does not currently take z coordinates for the start and end points, it should be modified to do so. There are two ways to modify the line drawing — modifying the Bresenham algorithm, or scrapping the whole thing for a DDA algorithm. Both approaches are about the same complexity, but based on some other changes that will occur in the next assignment, a DDA approach is highly recommended.

  The DDA approach is discussed in class and involves adding additional variables to store the increment in z and track the increasing (or decreasing) value of z from one endpoint to the other.

  Modifying the Bresenham routine involves computing an incremental change in z from one end of the line to the other by taking the change in z and dividing by the number of steps taken by the line (dx or dy depending on whether the line is mostly horizontal or vertical). This increment must be a floating point value. A z variable is initialized to the z value of the starting endpoint and incremented with each step of the line until the ending z is reached.

  Both the DDA and Bresenham routines need to take care of the special case of a line segment viewed end on, which projects to a point. In such a case, the smallest z value of the endpoints is used to compare against the z buffer. When the line is drawn, the z value of the closest point is placed in the depth buffer.

Pixel Plotting

All routines that plot pixels should be modified to use the depth buffer. Every time a pixel is to be plotted, its depth should be compared with the value in the depth buffer. Rather than doing all this code modification in your line drawing routine, point drawing routine, circle drawing routine, and anything else that needs to call `rd_write_pixel()`, make your own routine for drawing a pixel. It should do the zbuffer testing and call `rd_write_pixel()`. Your primitive drawing routines should then call your pixel drawing routine instead of calling `rd_write_pixel()` directly. This will also be a good preparation for the next assignment where even more things will need to be calculated for each pixel to be drawn.

Objects

- Make sure your `cube`, `cylinder`, and `cone` routines send polygons through the pipeline instead of sets of disconnected line segments. The vertices of each polygon should be traced in counter clockwise order as seen from the outside. This is in preparation for the next assignment, and will not be tested here. (In other words, if you don't do this now, no one will know, but you'll be hating life in the next assignment.)
- If you have not already done so, change your `sphere` routine so that it will draw a polygonalized version of the sphere. Use the latitude/longitude algorithm algorithm discussed in class. Also modify your routine so that it honors the *radius* parameter described in the RD command language.

Pass off

At pass off time, show your system running your version of the test scene files. Be prepared to modify the scene files and regenerate images during the pass off.

As always, make a scene of your own, and show the resulting image at passoff.

Wrap up the project into a single file and submit it according to the instructions given on the course web site.